



**FACULTAD DE
INGENIERÍA**

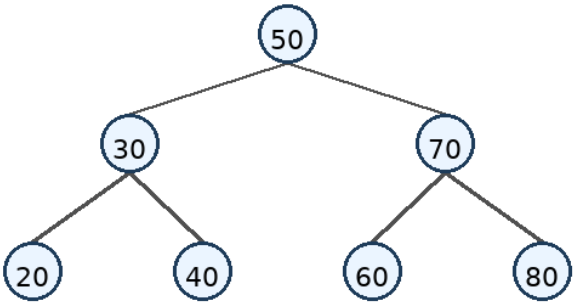
UNIVERSIDAD DA VINCI
DE GUATEMALA

ESTRUCTURA DE DATOS
NOMBRE: CARLOS EMILIO LIMA ORDOÑEZ
CARNÉ: 202500147

Fichas Técnicas de Estructuras Arbóreas e Implementación de Árbol Binario de Búsqueda

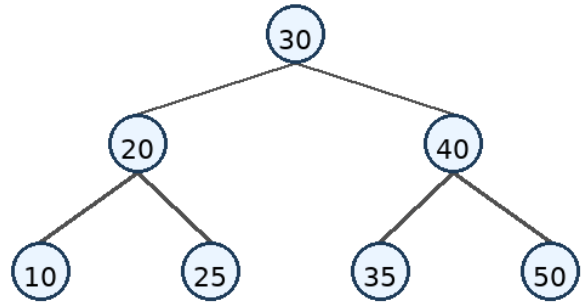
Árbol Binario de Búsqueda (BST)

Arbol Binario de Busqueda (BST)



Componente	Descripción
Identificación	Estructura jerárquica donde cada nodo tiene como máximo dos hijos. Los valores menores se ubican a la izquierda y los mayores a la derecha.
Complejidad algorítmica	Búsqueda: promedio $O(\log n)$, peor caso $O(n)$ Inserción: promedio $O(\log n)$, peor caso $O(n)$ Eliminación: promedio $O(\log n)$, peor caso $O(n)$
Características mecánicas	Se organiza comparando cada valor con la raíz. Si es menor avanza al subárbol izquierdo; si es mayor, al derecho. Mantiene orden lógico, pero no se balancea automáticamente.
Ventajas y limitaciones	Ventaja: permite búsquedas ordenadas y recorridos ascendentes simples. Limitación: si se insertan datos ordenados, puede degenerarse en una lista y perder eficiencia.
Caso de uso en la industria	Puede utilizarse en índices simples, sistemas de búsqueda en memoria y estructuras base para explicar motores de búsqueda o almacenamiento ordenado.

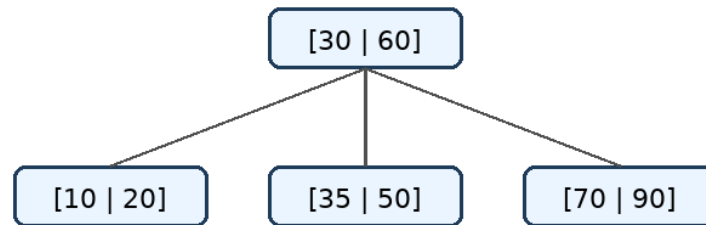
Arbol AVL balanceado



Componente	Descripción
Identificación	Variante de árbol binario de búsqueda que mantiene balanceada la altura de sus subárboles mediante rotaciones.
Complejidad algorítmica	Búsqueda: promedio $O(\log n)$, peor caso $O(\log n)$ Inserción: promedio $O(\log n)$, peor caso $O(\log n)$ Eliminación: promedio $O(\log n)$, peor caso $O(\log n)$
Características mecánicas	Cada nodo controla un factor de balance entre el subárbol izquierdo y derecho. Cuando la diferencia de alturas excede el límite permitido, se aplican rotaciones simples o dobles.
Ventajas y limitaciones	Ventaja: garantiza operaciones rápidas aun con muchos datos. Limitación: requiere más trabajo en inserciones y eliminaciones porque debe recalcular balance y rotar nodos.
Caso de uso en la industria	Se usa en aplicaciones donde se necesita consulta rápida en memoria, como tablas de símbolos, índices internos y estructuras de búsqueda ordenada.

Árbol B / B+

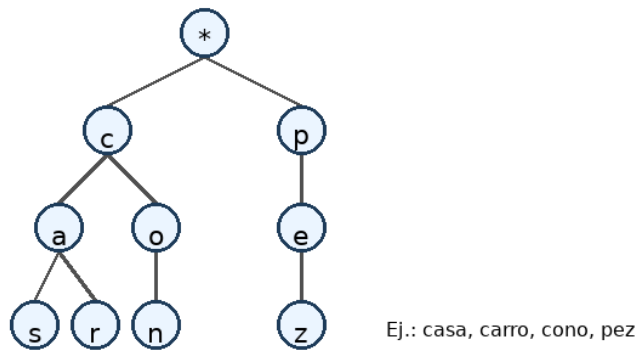
Arbol B / B+ con multiples claves



Componente	Descripción
Identificación	Árbol balanceado de múltiples caminos diseñado para almacenar grandes cantidades de datos y reducir accesos a disco.
Complejidad algorítmica	Búsqueda: promedio $O(\log n)$, peor caso $O(\log n)$ Inserción: promedio $O(\log n)$, peor caso $O(\log n)$ Eliminación: promedio $O(\log n)$, peor caso $O(\log n)$
Características mecánicas	Cada nodo puede guardar varias claves y tener varios hijos. En B+ las claves de datos suelen estar en las hojas, y las hojas pueden enlazarse para recorridos por rango.
Ventajas y limitaciones	Ventaja: excelente para bases de datos y sistemas de archivos por su alto factor de ramificación. Limitación: es más complejo de implementar que un BST o AVL.
Caso de uso en la industria	Es fundamental en motores de bases de datos como MySQL, PostgreSQL y sistemas de archivos para índices y búsquedas por rango.

Trie (Árbol de Prefijos)

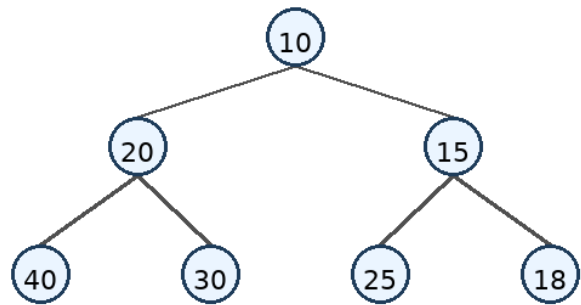
Trie: almacenamiento por prefijos



Componente	Descripción
Identificación	Estructura que almacena cadenas dividiéndolas carácter por carácter, compartiendo prefijos comunes.
Complejidad algorítmica	Búsqueda: promedio $O(m)$, peor caso $O(m)$ Inserción: promedio $O(m)$, peor caso $O(m)$ Eliminación: promedio $O(m)$, peor caso $O(m)$
Características mecánicas	Cada nivel representa un carácter de la palabra. Las rutas desde la raíz forman palabras completas o prefijos; m representa la longitud de la cadena.
Ventajas y limitaciones	Ventaja: muy eficiente para autocompletado y búsqueda por prefijos. Limitación: puede consumir mucha memoria cuando el alfabeto es grande o hay pocas palabras compartidas.
Caso de uso en la industria	Se usa en buscadores, correctores ortográficos, autocompletado, sistemas de diccionarios y filtrado de palabras.

Heap (Montículo Min/Max)

Heap Min: padre menor que sus hijos



Componente	Descripción
Identificación	Árbol casi completo que cumple la propiedad de prioridad: en Min-Heap el padre es menor que sus hijos; en Max-Heap es mayor.
Complejidad algorítmica	Búsqueda: promedio $O(n)$, peor caso $O(n)$ Inserción: promedio $O(\log n)$, peor caso $O(\log n)$ Eliminación raíz: promedio $O(\log n)$, peor caso $O(\log n)$
Características mecánicas	Normalmente se representa con arreglos. Al insertar, el valor sube si rompe la propiedad del heap; al eliminar la raíz, se reacomoda bajando el nuevo valor raíz.
Ventajas y limitaciones	Ventaja: permite obtener rápidamente el elemento de mayor o menor prioridad. Limitación: no está diseñado para búsqueda general de cualquier valor.
Caso de uso en la industria	Se utiliza en colas de prioridad, planificación de procesos, algoritmos como Dijkstra, HeapSort y sistemas de atención por prioridad.

Fase 2: Implementación Práctica en Java

La implementación desarrollada corresponde a un Árbol Binario de Búsqueda que almacena números enteros. No se utilizan librerías nativas de árboles; la estructura se programó desde cero mediante una clase Node interna.

El programa contiene inserción, búsqueda, eliminación y recorridos In-Order, Pre-Order y Post-Order. La clase Main realiza una demostración insertando siete valores, buscando valores existentes y no existentes, y eliminando nodos en los tres casos solicitados.

La eliminación se resolvió contemplando los tres escenarios: si el nodo no tiene hijos se elimina directamente; si tiene un hijo, se reemplaza por ese hijo; y si tiene dos hijos, se reemplaza por el menor valor del subárbol derecho para conservar la regla del BST.

Método	Función
insert(int value)	Inserta un valor en la posición correcta.
search(int value)	Devuelve true o false si el valor existe.
delete(int value)	Elimina un nodo considerando los tres casos críticos.
printInOrder()	Imprime los valores en orden ascendente.
printPreOrder()	Imprime primero la raíz.
printPostOrder()	Imprime la raíz al final.